

Microservices Basics

What are microservices?

Microservices are small, independent applications designed to perform specific business functions (like "payment processing" or "user management"). They run in their own processes and communicate with each other, making the overall system easier to build, update, and scale piece by piece.

How do microservices differ from monolithic architectures?

In a **monolith**, all code runs as a single, large application that shares one database and codebase. **Microservices** split the application into many small, distinct services, each having its own codebase and, often, its own database. Monoliths are simple to deploy initially, while microservices offer better fault isolation and independent scaling.

What are some benefits of using microservices?

Benefits include **independent deployment** (you can update one service without touching others), **better fault isolation** (if one service fails, the whole system doesn't crash), and the **freedom to use different technologies** (languages/databases) for different services based on their specific needs.

Can you mention any challenges you might face while working with microservices?

Challenges include **increased operational complexity** (managing dozens of services, networks, and databases), **distributed transaction management** (ensuring consistency across services), and **inter-service latency** (slow network calls between services).

How do microservices communicate with each other?

Microservices primarily communicate using lightweight protocols. This is typically done synchronously using **REST (HTTP)** for request-response interactions, or asynchronously using a **Message Broker** (like Kafka or RabbitMQ) for event-driven communication.

What is synchronous vs. asynchronous communication?

Synchronous communication means the sender waits for the receiver to process the request and send a response before continuing its work (e.g., a phone call). **Asynchronous communication** means the sender sends a message and continues immediately without waiting for a direct reply (e.g., sending an email).

What are some of the risks involved with inter-service communication?

Risks include **cascading failures** (where a slow service causes upstream services to fail), **network latency**, and **security vulnerabilities** (ensuring the requesting service is

authenticated and authorized). You must manage these risks using patterns like Circuit Breakers and central authentication.

API Gateway

What is the role of an API Gateway in microservices?

The **API Gateway** acts as the single point of entry for all client requests. Its main role is to route requests to the correct backend service. It acts as a **facade**, simplifying the client's interaction with the complex internal architecture.

How does an API Gateway manage traffic?

An API Gateway manages traffic by using **routing rules** based on the request URL, and by implementing **filters**. It can apply cross-cutting concerns like load balancing (distributing traffic), rate limiting (throttling requests), and monitoring before the request is forwarded to the correct service.

What are some security measures that can be implemented at the API Gateway?

Security measures include **Authentication** (checking the user's login credentials or token), **Authorization** (checking initial permissions), and **SSL/TLS Termination** (decrypting HTTPS requests). It centralizes these functions, offloading work from individual services.

Can you explain how an API Gateway can handle load balancing?

The API Gateway handles load balancing by maintaining a list of all available service instances. When a request comes in, the gateway uses an algorithm (like round-robin or least connections) to choose one healthy service instance and forwards the request to it.

Service Discovery

What is a Service Registry?

A **Service Registry** is a database or centralized server that stores the current network locations (IP addresses and ports) of all available microservice instances. It acts as a directory for the entire system.

How does service discovery work in microservices?

Service discovery allows a client service to find the network location of another service it needs to call. When a service instance starts, it **registers** its location with the Service Registry. A consuming service then **queries** the registry to get the address before making the request.

What would happen if a service registry fails?

If the Service Registry fails, **existing** connections between services would continue to work. However, no new service instances could register, and any service trying to find a **new** instance or a service that had just failed would be unable to locate it, leading to failures over time.

How do microservices update their registration and discovery information?

Services update their registration using a **heartbeat** mechanism. They periodically send a small signal to the Service Registry to prove they are still alive. If the registry doesn't receive the heartbeat within a defined time, it removes the service instance from its catalog.

Data Consistency (Basic)

What is eventual consistency?

Eventual consistency is a consistency model where, after an update, the data replicas across different services or databases will eventually become consistent, though there may be a temporary delay. It prioritizes availability and speed over immediate, strict consistency.

How do you handle data consistency in microservices?

Data consistency is typically handled using **asynchronous events** and the **Saga Pattern**. When one service changes its data, it publishes an event. Other services consume that event and update their local data accordingly, achieving eventual consistency.

What are the trade-offs of eventual consistency vs. strong consistency?

Eventual consistency provides **higher availability and lower latency** but risks temporarily displaying incorrect or stale data. **Strong consistency** (like traditional database transactions) guarantees that all users see the absolute latest data but increases latency and can hurt availability during network partitions or failures.

Deployment Basics

What are some strategies for microservices deployment?

Common strategies include **Blue-Green Deployment**, **Canary Releasing**, and **Rolling Updates**. These strategies aim to minimize downtime, reduce risk, and allow for quick recovery if a new deployment introduces bugs.

Can you describe blue-green deployment?

Blue-Green deployment involves running two identical environments: Blue (current live version) and Green (new version). When the new Green version is fully tested, traffic is instantly switched from Blue to Green. This allows for immediate rollback by switching traffic back to the stable Blue environment if any issues arise.

How does canary releasing differ from blue-green deployment?

Blue-Green switches all user traffic instantly from the old version to the new version. **Canary releasing** exposes the new version only to a very small subset of users (e.g., 5%). This allows the team to test the new version with real traffic and observe performance before rolling it out to everyone.

What tools would you recommend for automating microservices deployment?

I would recommend **Kubernetes (K8s)** for orchestration, **Jenkins** or **GitLab CI** for continuous integration/delivery, and **Helm** for packaging and managing the application configuration on K8s.

Monitoring & Security (Basic)

How do you monitor and manage microservices?

You monitor microservices by collecting **metrics** (CPU, memory, latency) and **logs** (application events) centrally. You manage them using **orchestration tools** (like Kubernetes) that handle scaling, health checks, and automatic restart on failure.

What metrics are important to monitor in a microservices architecture?

Important metrics include **Latency** (response time), **Throughput** (requests per second), **Error Rates** (failed requests), and **System Utilization** (CPU, memory, database connection pool usage).

What tools can be used for logging and monitoring in a microservices environment?

The standard tools for logging and monitoring are the **ELK Stack** (Elasticsearch, Logstash, Kibana) for centralized logging and **Prometheus/Grafana** for metrics collection and visualization.

How do you ensure security in microservices?

Security is ensured by centralizing authentication at the **API Gateway** and using **JWT (JSON Web Tokens)** for communication between services. You also enforce **TLS/HTTPS** for all network traffic and apply role-based access control (RBAC).

What are the common security patterns applicable in microservices?

Common security patterns include the **API Gateway pattern** (for centralized auth), the **JWT token pattern** (for stateless service-to-service communication), and the **Externalized Configuration** pattern (for securing secrets like database passwords).

Communication Patterns

What are the ways of communication between microservices?

The main ways are **Synchronous** (e.g., REST/HTTP and gRPC) for requests that need an immediate response, and **Asynchronous** (e.g., message queues like Kafka or RabbitMQ) for event notifications, streams, and delayed processing.

How would you call another service in the microservice architecture?

I would use a **Service Registry** (like Eureka or Consul) to find the target service's address. Then, I would use a library like **Spring WebClient** or **Feign Client** to make the synchronous REST call.

What are the trade-offs of synchronous vs. asynchronous communication between services?

Synchronous communication is simple, fast for simple requests, and provides immediate feedback. Its trade-off is **tight coupling** and the risk of **cascading failures**. **Asynchronous** communication promotes **loose coupling** and resilience but adds complexity and makes debugging harder due to delayed responses.

Simple Spring Cloud usage

How does Spring Cloud enhance microservices development in Spring Boot?

Spring Cloud enhances development by providing pre-built, production-ready solutions for common distributed systems problems. It offers easy-to-use patterns for **Service Discovery**, **Centralized Configuration**, **Client-Side Load Balancing**, and **Circuit Breakers**.

You are tasked with creating service-to-service communication, how does Spring Cloud assist?

Spring Cloud assists by providing **Declarative REST Clients** (like OpenFeign, which simplifies calling REST APIs) and integrating seamlessly with **Service Discovery** tools. This allows you to call another service by name (e.g., user-service) instead of using its physical IP address and port.

What are the components of a typical Spring Cloud architecture?

A typical Spring Cloud architecture includes a **Service Registry** (like Eureka) to track services, a **Configuration Server** to manage properties centrally, a **Gateway** (like Spring Cloud Gateway) for routing, and **Client Libraries** (like OpenFeign) for resilience and service lookups.

Docker / K8s Basic

What is the difference between Docker and Kubernetes?

Docker is a tool used for **containerization**; it packages an application and all its dependencies into a single, portable unit called a container image. **Kubernetes (K8s)** is an **orchestration platform** used for managing, deploying, and scaling those containers automatically across a cluster of machines.

If you were tasked with deploying a microservices application, how would you decide between Docker, Kubernetes, or both?

I would decide to use **both**. Docker is necessary for packaging each microservice. Kubernetes is necessary for managing the deployment, scaling, health checking, and networking between all the dozens of Docker containers in the ecosystem.

General Challenges

What is the disadvantage of microservices? What are the ways to address it?

The main disadvantage is **operational complexity**. Managing and monitoring many independent services, each with its own logs and databases, is much harder than a monolith. This is addressed by using **centralized logging/monitoring tools** (ELK, Prometheus), **automation (K8s)**, and **distributed tracing**.

What challenges have you faced while working with microservices, and how did you address them?

A common answer: I faced the challenge of **cascading failures** when one slow service brought down others. I addressed this by implementing the **Circuit Breaker** pattern (using Resilience4J) and adding proper **timeouts** to prevent upstream services from waiting indefinitely.

What security practices do you consider when developing microservices?

I consider using **TLS/HTTPS** for all inter-service communication to prevent man-in-the-middle attacks. I apply **role-based access control (RBAC)** to ensure services only access resources they need. And I store secrets (like database passwords) externally in a **Secret Management** tool (like Vault).

Are you familiar with any tools for monitoring the health and performance of microservices?

Yes, I am familiar with tools like **Prometheus** (for collecting metrics), **Grafana** (for visualizing dashboards), **Jaeger** or **Zipkin** (for distributed tracing), and the **Spring Boot Actuator** (for internal health checks and application metrics).